

并行计算 II 2026 春第四次作业

生命游戏并行模拟

王越洋

1 题目

实现二维生命游戏 (Conway's Game of Life) 的并行模拟。给定一个 $N \times N$ 的网格，每个格子只有两种状态：0 表示死亡，1 表示存活。

每一步迭代时，格子 (i, j) 的下一状态由周围 8 个邻居中的存活数量决定：

- 当前为 1，且活邻居数为 2 或 3：下一步仍为 1；
- 当前为 1，且活邻居数小于 2 或大于 3：下一步变为 0；
- 当前为 0，且活邻居数恰好为 3：下一步变为 1；
- 其他情况下一步为 0。

本作业采用固定死亡边界：网格外的所有格子均视为 0。

1.1 串行算法

```
for step = 1 to T do
  for i = 0 to N-1 do
    for j = 0 to N-1 do
      cnt = number of live neighbors around (i, j)
      next[i][j] = rule(current[i][j], cnt)
    end for
  end for
  swap(current, next)
end for
```

1.2 初始状态生成

为了便于自动化验证，本作业不使用随机数，而是用确定性公式生成初始网格：

```
grid[i][j] = 1 if ((i * 131 + j * 17 + 7) % 100) < 30
grid[i][j] = 0 otherwise
```

2 作业要求

- 阅读 `serial.cpp`, 理解串行生命游戏的计算、固定死亡边界和输出格式。
- 至少完成以下并行实现之一: `parallel_cuda.cu` (CUDA 单卡实现) 或 `parallel_mpi.cpp` (MPI 多进程实现)。
- 鼓励完成 MPI+CUDA 混合版本作为加分项, 但不强制。
- CUDA 实现建议: 一个线程负责一个格子; 使用两个网格数组做双缓冲; 每一步从 `current` 读、向 `next` 写, 然后交换指针; 注意边界格子的邻居计数。
- MPI 实现建议: 按行划分网格; 每个进程保存自己负责的行以及上下 ghost rows; 每一步先交换上下边界行, 再更新本地行; 最后将完整网格汇集到 rank 0 输出。

算例	N	T	主要考察目的
S	16	4	正确性 / 截图 / 可视化
M	257	100	非整除规模 / 自动验证
L	1024	200	性能与加速比测量

使用一张 GPU 即可完成 CUDA 实验。MPI 实验可使用 4 或 16 个 CPU 进程。

3 编译与运行

```
make
./serial 16 4
./parallel_cuda 16 4
mpirun -np 4 ./parallel_mpi 16 4
sbatch run.slurm
```

4 自动验证

`verify.py` 会用同样的公式重建初始网格, 串行迭代 T 步后逐格比对:

```
python3 verify.py life_N257_T100.txt
# PASS: final grid matches the reference for N=257, T=100.
```

输出格式为: 最终网格写入 `life_N<N>_T<T>.txt`, 第一行为 N N , 随后 N 行为最终网格, 每行 N 个 0/1 整数。

5 提交

提交内容包括: 修改后的并行代码; 算例 S 的运行截图和可视化图片; `verify.py` 验证算例 M 得到 PASS 的截图; 算例 L 的串行时间、并行时间和加速比; 报告。

邮件标题格式: 姓名-学号-第 4 次作业。截止时间 7 月 5 日。